

Warning! Engineered Linux Malware can Bypass Next-Gen Anti-Virus Solutions

Here's a quick demo of how anti-virus solutions let engineered Linux malware enter a protected system. The solution: build your own tools to test the security of the network and don't rely solely on automated anti-virus solutions.



Image Source: <https://www.freepik.com/>

A few weeks ago, I set out on a project that blended offensive security with a bit of creative engineering. The goal was simple but ambitious: to build a custom reverse TCP (Transmission Control Protocol) payload from scratch using Python, pack it into *(.elf)* executable, and test how stealthy it could really be against modern antivirus software. This was not just about gaining shell access. I wanted full remote control, including webcam snapshots, keylogging, screen capture, and file transfer capabilities. The idea was to explore, learn, and better understand both offensive and defensive security concepts through hands-on experimentation.

For most red teamers and cybersecurity hobbyists, tools like 'Msfvenom', 'Empire', 'TheFatRat', or 'Veil Evasion Framework' are the go-to options for obfuscation

and payload generation. These are powerful, but also extremely noisy. Modern next-gen antiviruses and EDR (Endpoint Detection and Response) solutions flag them almost instantly.

So, I decided to go custom, because:

- I could avoid signature-based detection.
- I would have complete control over every behaviour.
- I could better understand what is happening under the hood.
- I wanted to observe how detection engines really work.

Writing the Python scripts

To achieve this, I wrote two Python scripts.

The payload (target-side): This script connects back to the attacker's machine and executes commands received from it. It handles everything from command execution,